

Running a pces simulation

This document describes how one executes a set of **pces** simulation runs that together comprise an 'evaluation'. This document is a companion to others that together provide a picture of what **pces** comprises.

- [PCES-Introduction.pdf](#) and [PCES-Internals.pdf](#) describe the model used to describe workflows in **pces**, and the functions used to demark measurement points. A set of **pces** experiments computes and reports estimated performance metrics (like latency and throughput) observed between measurement points.
- [PCES-API.pdf](#) and [MRNES-API.pdf](#) document the formatting requirements of input files read by the **pces** simulation.
- [MRNES-Introduction.pdf](#) and [MRNES-Internals.pdf](#) describes the **mrnes** repository, which is imported by **pces** and provides the modeling support for describing the network and computing devices upon which the **pces** applicatin workflows executed.
- [xlsxPCES.pdf](#) describes a tool for building **pces/mrnes** models. This tool creates files that are directly used by the scripts we describe here. The repository <https://github.com/iti/pcesapps> contains a number of examples of applications that can be built and run using xlsxPCES. See also [PCES-Apps.pdf](#).

github.com/iti/pcesapps

Distribution *github.com/iti/pcesapps* contains tools for running *pces* models, and directories that contain specification for example models.

The tools for running *pces* reside in subdirectory *pcesapps/simulator*. The files and directories in *pcesapps/simulator* include

- *sim-dir*, a subdirectory with .go code front-end for performing a simulation run
- *input*, a subdirectory where the simulator looks for its input files
- *template*, a subdirectory where the templated (i.e. symbol-bearing) versions of the simulation experiment are placed.
- *output*, a subdirectory where the output from a simulation run is placed
- *run.py*, a script that for each experiment creates the input files for the specified run, executes the run, and gathers the results of the run. At the end of the runs it forms an output file *output/results.yaml* that contains the measured and reported results of each run.

run.py expects the following input files, the formats for each having been described in companion API documents:

- *cp.yaml*, a description of the computational patterns, the functions they organize, and the functions' input/output relationships.
- *cplnit.yaml*, description of the configuration parameters for each of the model's computational functions.
- *topo.yaml*, a description of the model of computers and network on which the functions of the computational patterns are executed.

- *funcExec.yaml*, a table with execution timing information for the model's functions.
- *devExec.yaml*, a table with execution timing information for the operations performed by switches and routers.
- *exp.yaml*, description of performance parameters to ascribe to model components
- *map.yaml*, a mapping of each of the model's functions to one of the network model's computational devices.
- *experiments.yaml*, a description of the set of runs to perform on a model, where the values to assign to each free variable are given.

run.py

Performing an evaluation

On developing a model

The first step in performing an evaluation using **pces** is to develop a model. **pces** notions of functions and computational patterns are used to lay out chains of function evaluations which in aggregate capture the most significant (meaning here 'time-costly') operations that must be represented, and those operations whose performance is of particular interest as their configuration parameters are changed.

A TBD companion document will eventually lay out principles and identify low-level details to consider when developing a **pces** evaluation of a system, here we just point to *xlsxPCES* as a viable option to express and validate the correctness of a model to be simulated. Ultimately though, what *run.py* needs is for *simulator/input* to have correctly formatted versions of the eight input files identified earlier, whatever the source.

On executing a simulation run

The simulation runs can be performed in the host operating system of the user's computer, executing a binary compiled on that computer using a sufficient new version of the Go language, or can be executed by building and running a Docker container that has an internal version of the simulator, and works by importing the model input files and exporting the result files. Either on the host operating system or within a container, the execution of a simulation run is initiated with a command

```
% sim -is args/args-sim
```

The contents of the command line file *simuator/args/args-sim* include a number of commands that *run.py* writes in, depending on its own input arguments. The contents of *args-sim* also include selections made by the user, encode in file *simulator/args/args-sim-template*, described below.

command line argument	Explanation
-stop stoptime	The simulation stopping time (in units of seconds)
-rngseed initialSeed	An initial value (integer) to use in seeding a random number generator so as to produce reproducible behavior
-tunits unitcode	unitcode is in {'sec', 'msec', 'musec', 'nsec'} giving units of time in measurement latency reporting

Table 1: User specified command line arguments for simulation runs

These are the only parameters a user needs to be concerned with before launching an evaluation. The simulation activity will stop once there are no events on the event-list, meaning there is no starting of a new execution thread anytime in the future, and all previous execution threads have completed. The 'stoptime' then need only be large enough relative to the latest planned-for execution thread beginning to ensure that they have all started and completed. On the other hand, evaluations that are measuring throughput may be written to run traffic continuously through virtual time until the stoptime is reached.

Executable file 'sim' is a compiled version of the 'main' function of the simulator code (*simulator/sim-dir/sim.go*):

```
package main
import (
    "fmt"
    "github.com/iti/pces"
)

func main() {
    pces.ReadSimArgs()
    pces.RunExperiment(expCntrl, expCmplt)
    fmt.Println("Done")
}
```

The **pces** repository has a method (*pces.ReadSimArgs*) to read in the input files and parse them, and another (*pces.RunExperiment*) to run the experiment. The two input arguments to the latter function are themselves names of functions in the main module, but which appear in *simulator/sim-dir/exp.go*. The **pces** control code calls function *expCntrl* after parsing the input files but before beginning to execute events, as *expCntrl* is responsible for scheduling the start events.

```

func expCntrl(evtMgr *evtm.EventManager, context any, data any) any {
    // go through all comp patterns looking for functions of the 'start' class,
    // and schedule them for execution
    for _, cpi := range pces.CmpPtnInstByID {
        for _, cpfi := range cpi.Funcs {
            if cpfi.Class == "start" {
                evtMgr.Schedule(cpfi, &cpfi.Class,
                    pces.EnterFunc, vruntime.SecondsToTime(0.0))
            }
        }
    }
    return nil
}

```

The function signature of `expCntrl` is that of all event handling routings. The outer loop iterates over data structures that represent computational patterns, the inner loop iterates over all functions attached to the chosen computational pattern, and the test made is whether the class of the selected function is "start", a reserved keyword for one of the **pces** function classes. If the function matches this test, the scheduling method associated with the event list is called to execute 0.0 seconds into the future a function (`pces.EnterFunc`) that will determine when the selected start function actually is run first (based on its configuration parameters). So the simulation is seeded for activity by picking out the "start" functions and arranging to have the execution thread each starts scheduled.

After the simulation run terminates the method `pces.RunExperiment` causes `expCmplt` to execute. All it does is to save the simulation run's measurement outputs in a file that will be read by `run.py`.

The point of exposing `expCntrl` and `expCmplt` outside of the **pces** repository is to simplify modifications that might be desired for initializing the simulation run, or modify what is calculated and reported when the run has completed.

Perform an evaluation using *run.py*

One execution of `run.py` initiates multiple individual simulation runs, each run being initialized with a set of parameters. So, for instance, to observe the sensitivity of the latency between some particular source and destination as a function of the speed of device interfaces, one could run one experiment with 1Mbps interfaces, another with 10Mbps interfaces, and another with 100Mbps interfaces. The set of experiments to run is described in `experiments.yaml`, and `run.py` manages the setup, execution, and result-gathering for each.

File `simulator/args-run` is a file containing command-line arguments for `run.py`. These are given below

command line argument	explanation	required
-template templateDir	A subdirectory (nominally <i>pcesapps/simulation/template</i> in the repository, but can be configured) where input files are placed. Some of these files contain variable symbols that are instantiated before a simulation run.	Yes
-input inputDir	Names a subdirectory where a complete set of instantiated input files is placed by <i>run.py</i> in preparation for a simulation run. Nominally <i>pcesapps/simulator/output</i> in the repository, but can be configured.	Yes
-output outputDir	Names a subdirectory where the simulator writes its output, and <i>run.py</i> gathers and collates it, producing at the end of its execution a file <i>/output/results.yaml</i> .	Yes
-extern externDir	When present, means the simulator will be run from inside a Docker container, which links a file directory <i>/tmp/extern</i> inside of the container with a file directory outside of the container. The -extern value names the outside directory.	No
-container tag	When present, means the simulator will be run from inside a Docker container. The value of -container is the tag which identifies the container.	

Table 2: Command-line parameters for *run.py* script

As shown in *xlsxPCES*, a model can be expressed including substrings that are flagged as 'symbols' which before an experiment is run are replaced with actual values. *experiment.yaml* is formatted as a list of dictionaries, with each dictionary assigning a concrete value to every symbol. A symbol substring is flagged as a string whose first character is '\$'.

run.py is written in a way that regardless of which execution option is selected, when *run.py* starts running it assumes the input files are all in the subdirectory named by *run.py*'s '-template' argument.

The steps *run.py* takes to form the completed set of input files for each run and launch the run are as follows.

1. If the native operating system is to run the *sim* executable, check whether executable *simulator/sim-dir/sim* exists. If not, spawn a process that compiles *sim.go* and *exp.go* to create it. If the runs are to use a containerized version of *sim*, use Docker commands to look for the existence of a container with name 'tag', the value of the -container command line argument. If the container is absent a process is spawned to build it.
2. Copy all of the files in *simulator/template* into *simulator/input*.
3. For each experiment described in *simulator/template/experiments.yaml*
 - extract the dictionary that maps symbol codes to values, and the list of codes that identify which files the symbols are found in.

- Copy each of these files from *simulator/template* to *simulator/input*, and then for each symbol do a global replacement of the symbol with the string identified as its value for this experiment, in each of the files *experiment.yaml* implies may contain this symbol. After this step there are no symbols in any file located in *simulator/input*.
 - Create file *simulator/args/arg-sim*, in preparation for the simulation run.
 - Spawn a process that executes the simulator, using *simulator/args/args-sim* as the command-line file, *simulator/input* as the directory where input files are located, and *simulator/output* where output files will be located.
 - On completion of the run, gather the output in preparation for eventual aggregation.
4. Complete the evaluation by creating an output file *simulator/output/results.yaml* that describes all measurements made in each run.

simulator/output/results.yaml

As an example, the results reported from doing an evaluation on the 'running model' described in companion document 'xlsxPCES.md' is

```
pces evaluation run at time 2024-12:52:05.546541
- exprmnt: exp-1
  measurements:
  - index: 1
    latency: 1.0840000000007421 (msec)
    measurename: end2end
    waypoints: []
- exprmnt: exp-2
  measurements:
  - index: 1
    latency: 24.84399999999914 (msec)
    measurename: end2end
    waypoints: []
- exprmnt: exp-3
  measurements:
  - index: 1
    latency: 1.0790000000015425 (msec)
    measurename: end2end
    waypoints: []
- exprmnt: exp-4
  measurements:
  - index: 1
    latency: 24.838999999999942 (msec)
    measurename: end2end
    waypoints: []
```

Figure 1: Evaluation output from executing xlsxPCES running example

The output makes reference to experiments 'exp-1' through 'exp-4' that are described in input file *input/experiments.yaml*, which is part of the running example of a xlsxPCES model described in 'xlsxPCES.md'. Briefly, a packet is generated and encrypted by one device and then sent to another (through a switch) which decrypts its, processes it, encrypts the result, and returns the encrypted result to the origin where it is decrypted and the result processed. The main points to be understood here are that each of the experiments reports a latency between the start and end measurement endpoints, at the origin. These points are configured with the identifier 'end2end'. In this example the latency is reported in milliseconds, the units used in the report is a **pces** run-time parameter.

pces allows for multiple measurements within a single simulation run, between the same two measurement points and/or between multiple pairs of measurement points. A given pair of points will share the same 'measurename' attribute as this is a measurement point identifier; each initiation of a measurement using the same beginning measurement point increases the 'index' attribute.

As described in 'xlsxPCES.md', the **pces** user has the flexibility of selecting the parameters to be varied each run and the values given the experimental parameters. The user's knowledge of that setup is required then to take the measurements *run.py* causes to be reported and organize them in tables, interpret them, graph them, whatever the end result the user has in mind for these results. The results file is written out in yaml format, which supports a script-based approach to analyzing them.